

Enhancing the Search for Online Baking Recipes Using an LLM Agent Framework

Anna Li (tpb3rw) and Caroline Gihlstorf (czm5kz)

1 Motivation

Searching for baking recipes online can be time-consuming. A single search query outputs too many results to review in a reasonable amount of time, and many websites claim to have the “best” recipe, leaving it up to you to go through each recipe manually to decide which one to bake. People also have different preferences regarding the texture, taste, and types of ingredients to use for a particular baked good (e.g., chewy vs crunchy chocolate chip cookies, dairy-free recipes, etc), but not everyone has enough food science knowledge to know which combinations and ratios of ingredients are needed to get their desired outcomes. Even if someone were to spend time filtering through a set of search results, they might not be able to determine whether the types or ratios of ingredients in the recipe will give them the taste and texture that they want.

To solve this problem, we implement an additional backend system on top of a simple internet search that automatically filters through baking recipes and scores them based on how well they match the search query. Recipes can be filtered based on requirements that certain ingredients be included or left out (e.g., use dairy-free milk, does not use dutch-processed cocoa powder). Recipe scores evaluate how the ingredients in the recipe contribute to the taste and texture of the baked good. We use a combination of LLMs and deterministic scripts to filter, score, and explain the reasoning behind the final list of recipes returned by the system. With our tool, we hope that searching for baking recipes becomes less time-consuming and more attuned to people’s particular preferences.

2 Methods

Our pipeline has three main components: query generation and understanding, recipe search and parsing, and recipe scoring.

2.1 Query Generation and Understanding

Given an initial input into our system, we prompt an LLM (GPT-4.0 mini) to generate a set of queries suitable for an internet search based on the input. We first ask the LLM to categorize the input based on the type of baked good that will be made and list any preferences to consider regarding desired ingredients, taste, and texture. We then ask the LLM to generate multiple queries based on the input to increase the diversity of results from the internet search. For example, given the input *I’m looking for a fudgy, chewy brownie recipe with a rich chocolate flavor, that does not use dutch-processed cocoa powder*, the LLM outputs the following set of categories and preferences:

- **Category:** brownie
- **Constraints:** contains chocolate, no dutch-processed cocoa
- **Preferences:** fudgy texture, chewy texture, rich flavor

From these specifications, the LLM generates the following queries:

1. *fudgy brownie recipe natural cocoa*
2. *chewy brownie recipe rich chocolate flavor no dutch-processed cocoa*

2.2 Recipe Search and Parsing

Given a set of LLM-generated search queries, we then prompt the LLM to use the DuckDuckGo (DDG) internet search tool to search for recipes online using the queries. We implement a domain cap on the number of recipes that can be taken from a single website to ensure that multiple different recipe sites are represented in our results. Once we get the results from our DDG search, we take a snippet of each returned recipe and compute its cosine similarity with the query. From there, we choose the top most semantically similar snippets. For each snippet, we then retrieve the full recipe and ask the LLM to parse it to get information about the ingredients needed, the amounts of each ingredient needed, the number of steps needed to bake the recipe, and any specific techniques that are mentioned. This information will be used to determine the score of each recipe in the next step. If the LLM is unable to parse the recipe, we simply skip the scoring step and return the most similar results.

2.3 Recipe Scoring

We want to know how well each recipe aligns with the initial search queries in terms of the specific taste and/or texture specified, considerations for gluten-free binding agents, and how accessible the recipe is to make (less accessible recipes require more specialized techniques or equipment to make, more accessible recipes do not). Taste and texture are scored by scanning through the recipe’s ingredients and determining how much the ratios and amounts of different ingredients contribute to factors such as chewiness, moisture, and sweetness. We use a deterministic script to calculate recipe scores. Once the recipes are scored, we prompt the LLM to adopt the role of a food scientist to explain why each recipe received the score that it did. We then output the highest scored recipes along with their scores and explanations.

Our system is also equipped with a chat feature built using LangGraph that allows for further interaction after an initial recipe search is completed. For example, you may want to ask the model to find recipes that use a specific type of ingredient, clarify information about the retrieved recipes, or modify your search to include recipes that have a different taste or texture than what was specified in the original input. For example, after initially inputting *I’m looking for a fudgy, chewy brownie recipe with a rich chocolate flavor, that does not use dutch-processed cocoa powder*, you could ask the system whether any of the returned recipes contain espresso powder. If not, you could ask the system to search for recipes that do contain espresso powder. A mermaid diagram of the LangGraph structure behind this system is displayed in Figure 1.

2.4 User Interface

To make it easier to interact with our system, we build a webpage that allows you to query our system through an intuitive user interface. An image of the main page of our web interface can be found in Figure 2. After submitting an initial query, the system processes the input and returns a set of recipes with scores and their corresponding explanations. You can also use the search bar to further communicate with the system after receiving initial results, for example, to ask whether any of the recipes contain a certain ingredient or to ask for new recipes with a particular ingredient substitution. Our web interface also includes a feature that takes in the URL of a recipe, parses it, and assigns it a score and an explanation. This is useful if you have already have a recipe, but still want to know how the ingredients and their ratios will affect the taste and texture of the baked good. We also offer an option to save recipes to a “recipe box” for quick, easy access to all your favorite recipes. A demo video of our system can be found at <https://drive.google.com/file/d/1ATQd0tkALzxMcDRieISfMvUGNhY6Moft/view>.

3 Evaluation

We asked two participants to watch our demo video and give their thoughts on the pros and cons of our system. One participant highlighted that the explanations given with each score would make them feel more confident in the model’s choice of recipes. Another participant commented on the benefits of being able to personalize recipes to your particular preferences in the input query. One con noted by the participants was that the scoring mechanism was unclear in that they didn’t know whether a score of 84 was expected to be a good recipe or a mediocre recipe. One participant also noted that people might not always know their

preferences, making personalization more difficult. Other comments we received related to the user interface of our website – for example, that there was a decent variety of dietary restrictions to select from and that it was useful to be able to store your recipes, but that an expanded list of dietary selections (e.g., listing specific types of nuts to avoid as opposed to a general “nut-free” option) and more explanation of what each dietary restriction means (e.g., what “keto” means) would be helpful.

From our participant feedback, our system seems to have potential for personalized recipe searches, but could benefit from some adjustments to make the scoring mechanism more clear and the user interface more accommodating to a larger and more specific set of dietary restrictions.

4 Conclusions

We build a system that uses an LLM agent to search and parse baking recipes, score the recipes using deterministic scripts, and generate LLM explanations for each score. Feedback from two participants highlights the potential of our system to be a useful, personalizable recipe search assistant but notes some adjustments to be made to the system and the web interface to be more intuitive and navigable.

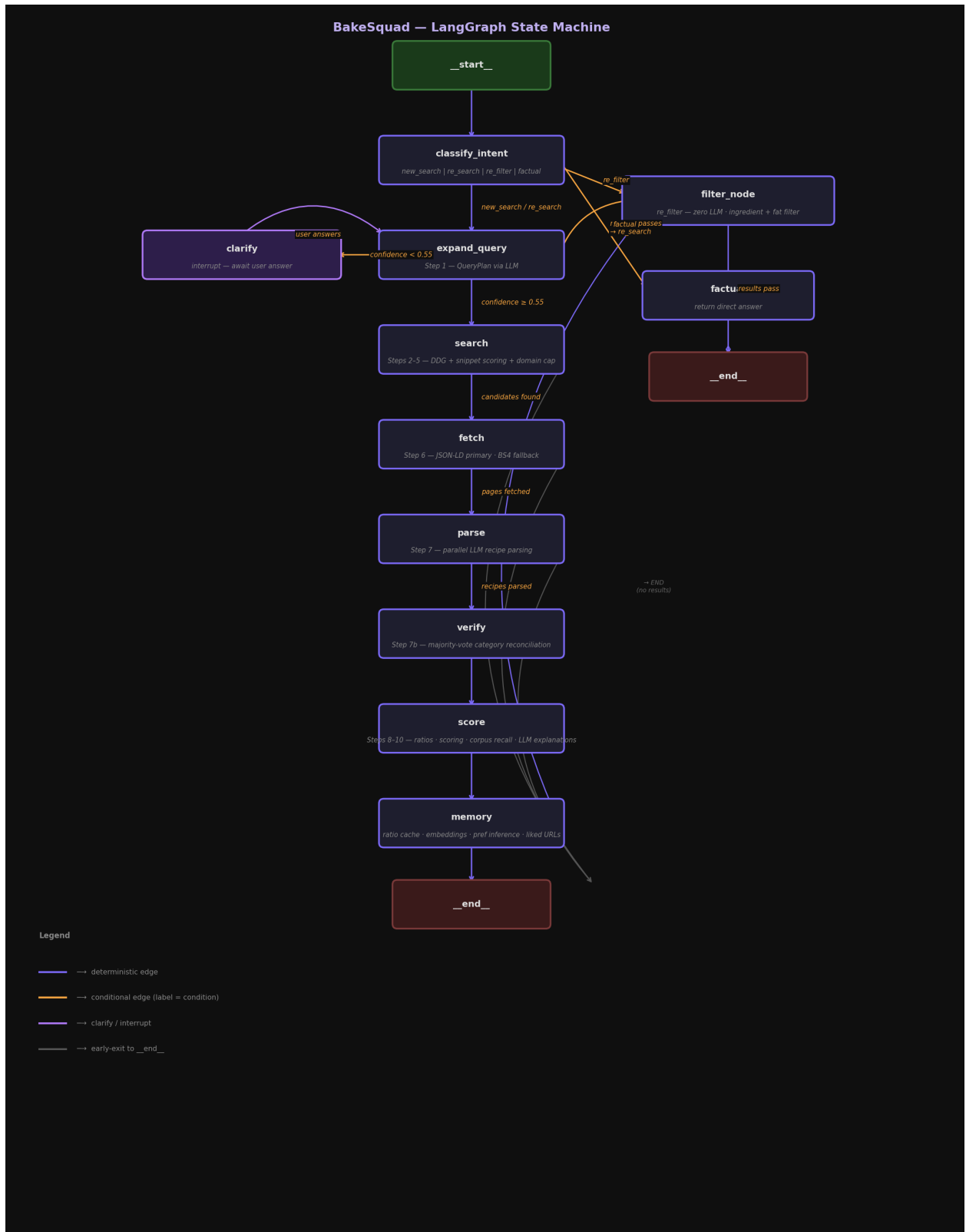


Figure 1: Mermaid diagram for our LangGraph backend.

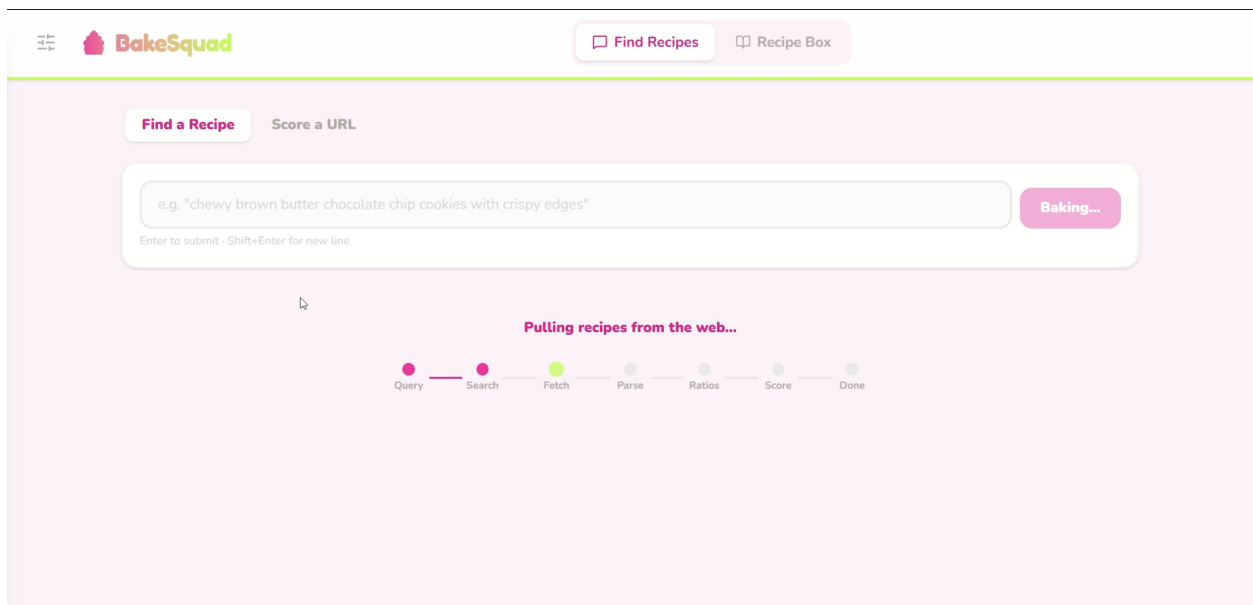


Figure 2: The main page of the web interface for our system.